

Morning Launcher

— Capstone Part 1

One command fires a whole morning routine: a notification, the clipboard, scheduling — the scaffold before app control.

Where we are

Days 15-19 built isolated tools — decorators, the subprocess/osascript OS bridge, pyautogui, global hotkeys, screen recognition. Day 20 starts wiring them into one program: a launcher that runs a named routine.

BEFORE

Every morning you open the same tabs, copy the same boilerplate, set the same reminders — by hand, one action at a time.

AFTER

`python launcher.py work` → one routine fires a notification, loads your clipboard, and (Day 21) opens tabs + music. Config lives in JSON, not code.

Talking back: mac notifications

```
import subprocess

def notify(title, message):
    script = f'display notification
              f"{message}" with title "{title}"'
    subprocess.run([
        "osascript", "-e", script])

notify("Launcher", "morning routine started")
```

What this page does

- macOS has no built-in Python notification call. The trick: shell out to `osascript`, the command-line AppleScript runner.
- **`subprocess.run`** launches a separate program (here `osascript`) and waits for it to finish.
- **`display notification`** is the AppleScript command for a native banner.
- The **`-e`** flag means “run this one line of script,” no `.scpt` file needed.

⚠ Sender label

The banner shows up under the app that ran it (often “Script Editor”). Verify on your machine — I’m not certain of the exact label on your macOS version.

Reading & writing the clipboard

```
# pip3 install pyperclip  
import pyperclip  
  
# put text ON the clipboard  
pyperclip.copy("standup notes:")  
  
# read whatever IS on the clipboard  
text = pyperclip.paste()  
print(text)
```

OUTPUT

standup notes:

What this page does

pyperclip is a tiny cross-platform library that reads and writes the system clipboard — the same buffer Cmd+C / Cmd+V use.

- **copy(s)** replaces the clipboard with string s.
- **paste()** returns the clipboard's current text.
- In the launcher, a routine can pre-load a snippet you'll paste right after it runs.

Text only

pyperclip handles plain text, not images or files. For those you'd need a platform-specific API — out of scope here.

Running jobs on a timer

```
import schedule, time

def job():
    notify("Launcher", "time to study")

schedule.every().day.at("07:00").do(job)
schedule.every(30).minutes.do(job)

while True:
    schedule.run_pending()
    time.sleep(1)
```

What this page does

schedule lets you say “run this function every day at 7am” in plain English, instead of doing raw time math.

- **every().day.at(...)** registers a job; it does not run yet.
- **run_pending()** checks the clock and fires anything due — you must call it in a loop.
- **sleep(1)** stops the loop from pinning a CPU core.

⚠ Not a daemon

schedule only runs while your loop is alive. Close the terminal and jobs stop. Real scheduling needs launchd / cron — later.

Routines live in JSON, not code

```
# routines.json
{
  "work": {
    "notify": "time to work",
    "clipboard": "standup notes:"
  },
  "study": {
    "notify": "study mode on",
    "clipboard": ""
  }
}
```

What this page does

Each routine is a named block of settings. Storing them in a JSON file means you add or edit a routine by editing data — the Python never changes.

```
import json
def load(path):
    with open(path) as f:
        return json.load(f)
```

json.load() parses the file into a Python dict.

launcher.py v1 — the dispatch

```
def run_routine(name, routines):
    r = routines.get(name)
    if r is None:
        raise KeyError(f"no routine {name!r}")
    notify("Launcher", r["notify"])
    if r.get("clipboard"):
        pyperclip.copy(r["clipboard"])

if __name__ == "__main__":
    routines = load("routines.json")
    run_routine(sys.argv[1], routines)
```

What this page does

This is the whole v1 in one function: look up the routine by name, then perform its actions. No app control yet — that's Day 21.

- **.get(name)** returns None instead of crashing on a bad name — we raise a clear error ourselves.
- **sys.argv[1]** is the word you typed after the filename.

OUTPUT

```
$ python launcher.py work
→ banner: time to work
→ clipboard set
```

What each tool gives you

#	Tool	Does what	Key call
1	subprocess + osascript	native mac notification	<code>subprocess.run(["osascript","-e",...])</code>
2	pyperclip	read / write clipboard	<code>pyperclip.copy()</code> / <code>.paste()</code>
3	schedule	run jobs on a timer	<code>every().day.at("07:00").do(job)</code>
4	json	load routines from a file	<code>json.load(open(path))</code>

T O D A Y ' S E X E R C I S E · 7 5 M I N

Build launcher.py v1

Write a launcher that loads routines.json and runs a named routine's notification + clipboard actions. No tabs or music yet — prove the scaffold works end to end.

STRUCTURE

routines.json with at least 2 routines (work, study). launcher.py reads it with json.load.

FUNCTIONS

notify(title,msg) via osascript.
load(path). run_routine(name, routines) dispatches.

BEHAVIOR

python launcher.py work fires the banner + sets clipboard. Bad name raises a clear KeyError.

*Stretch: add a third routine **gaming**, and make a missing routines.json fail with a friendly message.*

Things that will trip you up

⚠ Quoting in osascript

The AppleScript string needs its own double quotes around {title} and {message}. An f-string inside single quotes keeps them. Test with a message that has an apostrophe — it may break.

💡 Install with pip3

pip3 install pyperclip schedule. Both are pure-Python and small. osascript is already on macOS — nothing to install for notifications.

⚠ schedule needs the loop

Registering a job does nothing on its own. If jobs never fire, you forgot the while True / run_pending() loop or the script already exited.

💡 Keep data out of code

Resist hard-coding routines as Python dicts. Editing routines.json is faster and won't break the program — this same split powers CRUX's config later.

The scaffold is the point

DAY 21 — TOMORROW

Wire in the real actions: Safari tabs + Spotify via osascript. work / study / gaming each do something visible.

WEEK 5 — NEWS AGENT

Same dispatch shape: a named task triggers a chain of steps. Swap notifications for fetch → summarize.

CRUX TOOL-USE

run_routine is a baby version of CRUX picking a tool by name. JSON config and the osascript bridge both carry forward.

Today you stopped writing scripts and started writing a system — something with config, a dispatcher, and room to grow.